CardDemo daily transaction report: wiki pages and I/O contract

Wiki pages generated by the FlowX.AI legacy transformation agent from AWS CardDemo (open source, Apache-2.0). CONTRACT.md is the input/output contract for the build.

Pages included:

- `02-processes/process-reports.md`
- `10-acceptance-criteria/acceptance-criteria-reports.md`
- `01-data-model/project-data-model.md`
- `03-ui-flows/ui-flow-reports.md`
- `04-integrations/system-statementpdf.md`
- `02-processes/process-date-utility.md`
- `CONTRACT.md`

`02-processes/process-reports.md`

Process: Reports

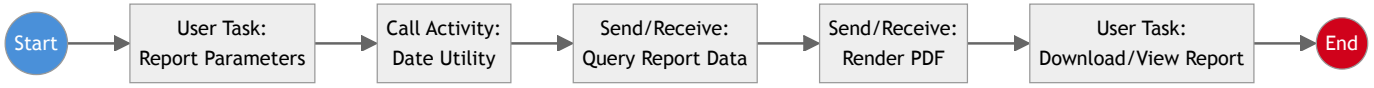
Powered by FLOWX.AI Coding Agents

Field	Value
Name	Reports
Type	Main Process
Legacy Source	`CORPT00C` / CR00 / BMS `CORPT00` (+ `TRANREPT.jcl`, `PRTCATBL.jcl`)
Status	Draft
Related UI Flows	ui-flow-reports
Related Data	Transaction, TransactionCategoryBalance

Business Description

Captures report parameters (date range, account, category) and generates a transaction report, optionally rendered to PDF. Legacy `CORPT00C` collects parameters and the `TRANREPT`/`PRTCATBL` jobs produce printed reports.

Process Flow



Nodes

#	Node	Type	Description	Legacy Reference
1	Report Parameters	User Task	Date range / filters	`RECEIVE MAP CORPT00`
2	Date Utility	Call Activity	Validate date range	`CSUTLDTC`
3	Query Report Data	Send/Receive Message Task	Aggregate transactions	`TRANREPT` logic
4	Render PDF	Send/Receive Message Task	Produce PDF	`TXT2PDF1.JCL`
5	Download/View	User Task	Deliver report	print/spool

Integration Touchpoints

Send Node	Topic	Payload	Receive	System
Query Report Data	`ai.flowx.in.carddemo.datastore.queryreport`	`{ from, to, filters }`	aggregate rows	FlowX Persistence
Render PDF	`ai.flowx.in.carddemo.pdf.render`	`{ reportModel }`	PDF ref	Statement PDF Renderer

Swimlanes

- **User, Admin**

Subprocess Calls

- Date Utility

---

*Powered by **FLOWX.AI** Coding Agents*

Acceptance Criteria: Reports

> **Powered by FLOWX.AI Coding Agents** · `@regression`

Verifies Reports (`CORPT00C`/CR00).

Configuration Dependencies

Config	Effect
`pdfRendererUrl`	PDF generation

Scenarios

#	Scenario	Steps	Expected Result
1	Generate report (happy)	Valid date range; Generate	Report produced, PDF offered
2	From > To date (validator)	from 2024-05-31, to 2024-05-01	Range validation error
3	Missing required date (validator)	Leave To empty	Required error
4	Empty result set (edge)	Range with no transactions	Report with zero rows, no error
5	PDF render failure (integration error)	Renderer unavailable	Error surfaced; retry offered `{{TODO: define retry}}`

Data Mapping Verification

Flow	Verify
Query Report Data	`{from,to,filters}` → aggregate rows
Render PDF	report model → PDF ref

Project Data Model — AWS CardDemo → FLOWX.AI

> **Powered by FLOWX.AI Coding Agents**

This section documents all **Project Data Types** derived from the CardDemo copybooks (`/app/cpy`). Each VSAM record layout maps to a Project Data Type; COBOL `05`-level items become attributes. Packed/zoned/signed numerics (`COMP-3`, `S9(n)V99`) map to `NUMBER`, fixed-width `X(10)` dates map to `DATE`, and FILLER tails are dropped. Enumerated reference codes are documented in Enumerations.

> **Type-conversion note:** `S9(10)V99` → `NUMBER` (decimal, scale 2); `9(n)` → `NUMBER` (integer); `X(n)` → `STRING` (maxLength n); `X(10)` date → `DATE`; `X(26)` timestamp → `DATE` (timestamp). The `ACCT-EXPIRAION-DATE` legacy typo is corrected to `expirationDate` — see `{{DECISION}}` below.

Business Categories

Category	Types
Core Processing	Account, Transaction, TransactionCategoryBalance
Domain	Customer, Card, CardXref
Reference / Config	TransactionType, TransactionCategory
Security	SecurityUser
Navigation (internal)	SessionContext

Account

Replaces the ACCOUNT-RECORD VSAM KSDS (`ACCTFILE` / `CVACT01Y`, 300 bytes). Key: `accountId`.

Attribute	Type	Required	Constraints	Description	Legacy Source
accountId	NUMBER	Yes	11 digits, unique key	Account identifier	`ACCT-ID 9(11)`
activeStatus	ENUM	Yes	AccountStatus	Active status flag	`ACCT-ACTIVE-STATUS X(1)`
currentBalance	NUMBER	Yes	decimal(12,2), signed	Current account balance	`ACCT-CURR-BAL S9(10)V99`
creditLimit	NUMBER	Yes	decimal(12,2), signed	Credit limit	`ACCT-CREDIT-LIMIT S9(10)V99`
cashCreditLimit	NUMBER	Yes	decimal(12,2), signed	Cash credit limit	`ACCT-CASH-CREDIT-LIMIT S9(10)V99`
openDate	DATE	Yes		Account open date	`ACCT-OPEN-DATE X(10)`
expirationDate	DATE	Yes		Account expiration date	`ACCT-EXPIRAION-DATE X(10)` *(legacy typo)*
reissueDate	DATE	No		Account reissue date	`ACCT-REISSUE-DATE X(10)`

Attribute	Type	Required	Constraints	Description	Legacy Source
currentCycleCredit	NUMBER	Yes	decimal(12,2), signed	Current-cycle credits	`ACCT-CURR-CYC-CREDIT S9(10)V99`
currentCycleDebit	NUMBER	Yes	decimal(12,2), signed	Current-cycle debits	`ACCT-CURR-CYC-DEBIT S9(10)V99`
addressZip	STRING	No	max 10	Account ZIP code	`ACCT-ADDR-ZIP X(10)`
groupId	STRING	No	max 10	Disclosure/interest group id	`ACCT-GROUP-ID X(10)`

`{{DECISION: Correct the legacy field name typo ACCT-EXPIRAION-DATE → expirationDate (recommended), or preserve the misspelling for round-trip fidelity? ETL must map the legacy column either way.}}`

Used by: Account View, Account Update, Bill Payment, Transaction Posting, Interest Calculation.

Customer

Replaces CUSTOMER-RECORD (`CUSTFILE` / `CVCUS01Y`, 500 bytes). Key: `customerId`.

Attribute	Type	Required	Constraints	Description	Legacy Source
customerId	NUMBER	Yes	9 digits, unique key	Customer identifier	`CUST-ID 9(9)`
firstName	STRING	Yes	max 25	First name	`CUST-FIRST-NAME X(25)`
middleName	STRING	No	max 25	Middle name	`CUST-MIDDLE-NAME X(25)`
lastName	STRING	Yes	max 25	Last name	`CUST-LAST-NAME X(25)`
addressLine1	STRING	Yes	max 50	Address line 1	`CUST-ADDR-LINE-1 X(50)`
addressLine2	STRING	No	max 50	Address line 2	`CUST-ADDR-LINE-2 X(50)`
addressLine3	STRING	No	max 50	Address line 3	`CUST-ADDR-LINE-3 X(50)`
addressStateCode	STRING	No	max 2	State code	`CUST-ADDR-STATE-CD X(2)`
addressCountryCode	STRING	No	max 3	Country code	`CUST-ADDR-COUNTRY-CD X(3)`
addressZip	STRING	No	max 10	ZIP code	`CUST-ADDR-ZIP X(10)`
phoneNumber1	STRING	No	max 15	Primary phone	`CUST-PHONE-NUM-1 X(15)`
phoneNumber2	STRING	No	max 15	Secondary phone	`CUST-PHONE-NUM-2 X(15)`
ssn	NUMBER	Yes	9 digits (PII)	Social security number	`CUST-SSN 9(9)`
govtIssuedId	STRING	No	max 20	Government-issued id	`CUST-GOVT-ISSUED-ID X(20)`
dateOfBirth	DATE	No		Date of birth	`CUST-DOB-YYYY-MM-DD X(10)`
eftAccountId	STRING	No	max 10	EFT account id	`CUST-EFT-ACCOUNT-ID X(10)`
primaryCardHolderInd	STRING	No	max 1	Primary-holder indicator	`CUST-PRI-CARD-HOLDER-IND X(1)`
ficoCreditScore	NUMBER	No	300–850	FICO credit score	`CUST-FICO-CREDIT-SCORE 9(3)`

{{TODO: SSN and govt id are PII — define masking/encryption and access restrictions in FLOWX.}}

Used by: Account View, Account Update, Authorization.

Card

Replaces CARD-RECORD (CARDFILE + CARDAIX / CVACT02Y, 150 bytes). Key: cardNumber; alternate index on cardAccountId.

Attribute	Type	Required	Constraints	Description	Legacy Source
cardNumber	STRING	Yes	16 chars, unique key (PCI)	Card number	CARD-NUM X(16)
cardAccountId	NUMBER	Yes	11 digits, AIX	Owning account id	CARD-ACCT-ID 9(11)
cvvCode	NUMBER	Yes	3 digits (PCI)	Card verification value	CARD-CVV-CD 9(3)
embossedName	STRING	Yes	max 50	Name embossed on card	CARD-EMBOSSED-NAME X(50)
expirationDate	DATE	Yes		Card expiration date	CARD-EXPIRAION-DATE X(10) *(legacy typo)*
activeStatus	ENUM	Yes	CardStatus	Card active status	CARD-ACTIVE-STATUS X(1)

{{TODO: cardNumber and cvvCode are PCI-sensitive — define tokenization/masking strategy.}}

Used by: Card List, Card View, Card Update, Card Security, Authorization.

CardXref

Replaces CARD-XREF-RECORD (XREFFILE + CXACAIX / CVACT03Y, 50 bytes). Links card ↔ customer ↔ account; alternate index on accountId.

Attribute	Type	Required	Constraints	Description	Legacy Source
cardNumber	STRING	Yes	16 chars, key	Card number	XREF-CARD-NUM X(16)
customerId	NUMBER	Yes	9 digits	Customer id	XREF-CUST-ID 9(9)
accountId	NUMBER	Yes	11 digits, AIX	Account id	XREF-ACCT-ID 9(11)

Used by: Account View, Card List, Transaction Add, Authorization.

Transaction

Replaces TRAN-RECORD (TRANFILE / CVTRA05Y, 350 bytes). Key: transactionId.

Attribute	Type	Required	Constraints	Description	Legacy Source
transactionId	STRING	Yes	16 chars, unique key	Transaction identifier	TRAN-ID X(16)

Attribute	Type	Required	Constraints	Description	Legacy Source
typeCode	ENUM	Yes	TransactionType	Transaction type code	`TRAN-TYPE-CD X(2)`
categoryCode	ENUM	Yes	TransactionCategory	Transaction category code	`TRAN-CAT-CD 9(4)`
source	STRING	No	max 10	Transaction source	`TRAN-SOURCE X(10)`
description	STRING	No	max 100	Transaction description	`TRAN-DESC X(100)`
amount	NUMBER	Yes	decimal(11,2), signed	Transaction amount	`TRAN-AMT S9(9)V99`
merchantId	NUMBER	No	9 digits	Merchant id	`TRAN-MERCHANT-ID 9(9)`
merchantName	STRING	No	max 50	Merchant name	`TRAN-MERCHANT-NAME X(50)`
merchantCity	STRING	No	max 50	Merchant city	`TRAN-MERCHANT-CITY X(50)`
merchantZip	STRING	No	max 10	Merchant ZIP	`TRAN-MERCHANT-ZIP X(10)`
cardNumber	STRING	Yes	16 chars	Card used	`TRAN-CARD-NUM X(16)`
originTimestamp	DATE	No	timestamp	Origination timestamp	`TRAN-ORIG-TS X(26)`
processTimestamp	DATE	No	timestamp	Processing timestamp	`TRAN-PROC-TS X(26)`

Used by: Transaction List, Transaction View, Transaction Add, Transaction Posting, Reports.

TransactionCategoryBalance

Replaces TRAN-CAT-BAL-RECORD (`TCATBALF` / `CVTRA01Y`, 50 bytes). Composite key: `accountId` + `typeCode` + `categoryCode`.

Attribute	Type	Required	Constraints	Description	Legacy Source
accountId	NUMBER	Yes	11 digits, key part	Account id	`TRANCAT-ACCT-ID 9(11)`
typeCode	ENUM	Yes	TransactionType, key part	Transaction type	`TRANCAT-TYPE-CD X(2)`
categoryCode	ENUM	Yes	TransactionCategory, key part	Category	`TRANCAT-CD 9(4)`
balance	NUMBER	Yes	decimal(11,2), signed	Category running balance	`TRAN-CAT-BAL S9(9)V99`

Used by: Transaction Posting, Interest Calculation, Reports.

SecurityUser

Replaces SEC-USER-DATA (`USRSEC` / `CSUSR01Y`). Key: `userId`. Target FLOWX model gates access with roles; plaintext password is replaced by IdP credentials.

Attribute	Type	Required	Constraints	Description	Legacy Source
userId	STRING	Yes	8 chars, unique key	User id	`SEC-USR-ID X(8)`
firstName	STRING	Yes	max 20	First name	`SEC-USR-FNAME X(20)`
lastName	STRING	Yes	max 20	Last name	`SEC-USR-LNAME X(20)`
password	STRING	—	**DO NOT MIGRATE**	Plaintext password (legacy debt)	`SEC-USR-PWD X(8)`
userType	ENUM	Yes	UserType	Admin (`A`) / User (`U`)	`SEC-USR-TYPE X(1)`

`{{DECISION: Do NOT migrate the plaintext password column. Provision users in a modern IdP and map userType → FLOWX roles. Confirm IdP (Keycloak/other).}}`

Used by: Signon, User Administration.

SessionContext

Internal process data replacing the navigation portion of `CARDDEMO-COMMAREA` (`COCOM01Y`). Not persisted — lives in process instance data. Documented here for traceability; in FLOWX most of these become implicit process state.

Attribute	Type	Description	Legacy Source
userId	STRING	Signed-on user id	`CDEMO-USER-ID X(8)`
userType	ENUM	UserType	`CDEMO-USER-TYPE X(1)` (88 A/U)
selectedCustomerId	NUMBER	Current customer context	`CDEMO-CUST-ID 9(9)`
selectedAccountId	NUMBER	Current account context	`CDEMO-ACCT-ID 9(11)`
selectedCardNumber	STRING	Current card context	`CDEMO-CARD-NUM 9(16)`

`{{DECISION: The from/to tranid/program and last-map/mapset fields (CDEMO-FROM-*, CDEMO-TO-*, CDEMO-LAST-MAP) are pure navigation plumbing — replace with FLOWX process transitions and do not model them as data. Confirm.}}`

See Data Model Diagram for relationships.

---

*Powered by **FLOWX.AI** Coding Agents*

UI Flow: Reports

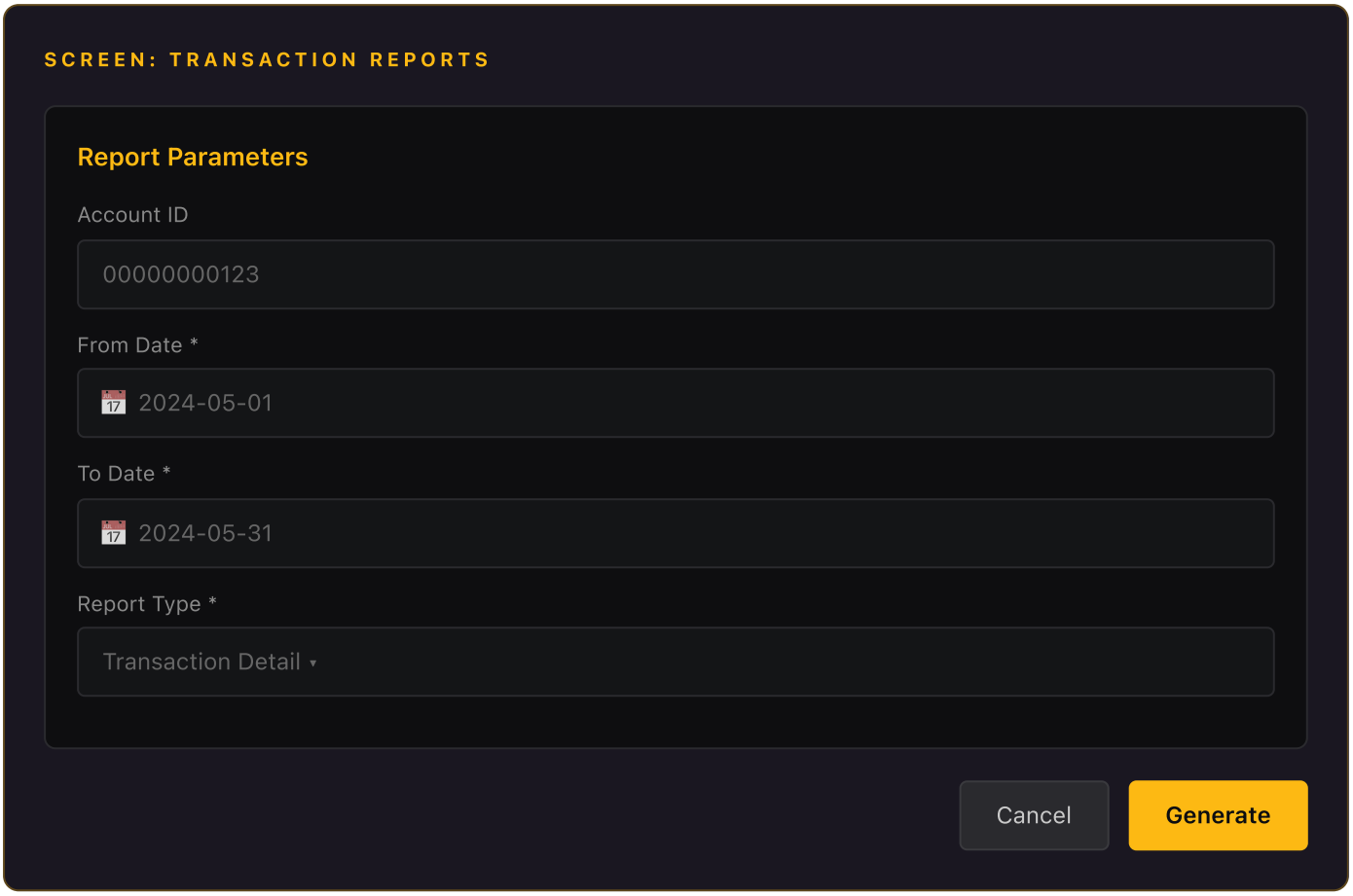
> **Powered by FLOWX.AI Coding Agents**

Field	Value
Name	Reports
Legacy Map	`CORPT00` / CR00
Process	Reports
Swimlane	User, Admin

User Story

As a clerk, *I need to* generate a transaction report for a date range *so that* I can review activity and produce a PDF for records.

Wireframe



Key Interactions

- **Generate** — query data, render PDF (via Statement PDF Renderer), offer download.

Data Bindings

Component	Data Path	Validator
Account ID	`report.accountId`	numeric
From/To Date	`report.fromDate` / `.toDate`	required, valid range (from ≤ to)
Report Type	`report.type`	required

---

*Powered by **FLOWX.AI** Coding Agents*

System: Statement PDF Renderer

> **Powered by FLOWX.AI Coding Agents**

Field	Value
Name	Statement PDF Renderer
Type	REST
Legacy Source	`TXT2PDF1.JCL` (TXT2PDF utility converting report text to PDF)
Auth	NO_AUTH `{{TODO: confirm}}`
Status	Draft

Purpose

Renders statement/report models to PDF, replacing the mainframe `TXT2PDF` utility. Used by Statement Generation and Reports.

`{{DECISION: Use the FlowX document plugin (recommended — see document-strategy.md) vs an external PDF microservice. If using the document plugin, this "system" is replaced by a Document Template and no external REST system is needed.}}`

Connection

Field	Value
Base URL	Configuration Parameter `pdfRendererUrl` (Environment Variable)
Operation	`POST /render` → `{ model }` → PDF binary/ref

Operations

Operation	Used By Workflow
Render PDF	Render Statement PDF

Process: Date Utility (Call Activity)

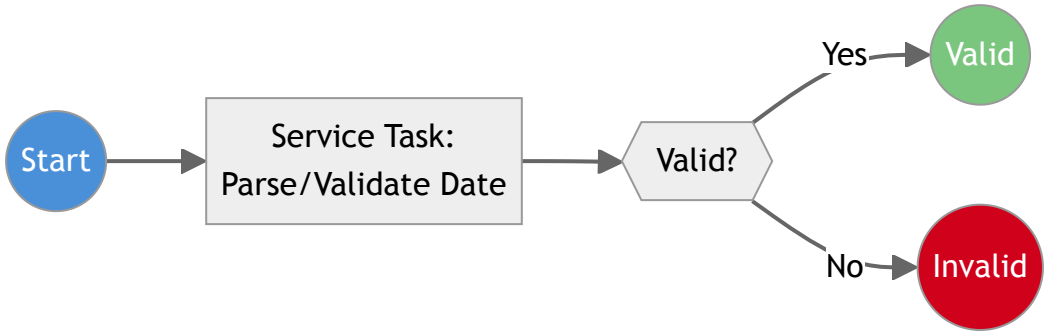
Powered by FLOWX.AI Coding Agents

Field	Value
Name	Date Utility
Type	Call Activity (reusable subprocess)
Legacy Source	`CSUTLDTC` (+ Assembler `COBDATFT`)
Status	Draft
Parent Processes	Transaction Add, Reports, batch pipelines

Business Description

Reusable date-format conversion/validation routine. Legacy CardDemo stores dates as `X(10)` strings and relies on `CSUTLDTC` / Assembler `COBDATFT` for conversion. In FLOWX most date handling is native; this call activity is retained only where legacy date semantics must be reproduced.

Process Flow



Nodes

#	Node	Type	Description	Legacy Reference
1	Parse/Validate Date	Service Task (NoOpHandler)	Convert/validate date string	`CSUTLDTC` / `COBDATFT`
2	Valid?	Exclusive Gateway	Verdict	—

Data Mapping

INPUT: `{ dateString, format }` · **OUTPUT:** `{ isoDate, valid }`

`{{DECISION: In most cases replace with the platform date library inline (no call activity). Keep this call activity only if legacy date edge-cases (e.g., Gregorian/Lilian conversions in`

`COBDATFT) must be preserved exactly.}}`

---

*Powered by **FLOWX.AI** Coding Agents*

Candidate I/O contract

What a migrated implementation must produce so the scorer can compare it with the oracle. The contract fixes **formats only**. It says nothing about behaviour, so it may be handed to a builder without leaking the answers.

Inputs (given per run)

The same files the COBOL chain reads, one record per line, fixed width, layouts as in the copybooks:

File	Copybook	Record
`dailytran.dat`	CVTRA06Y	350
`acctdata.dat`	CVACT01Y	300
`cardxref.dat`	CVACT03Y	50
`tcatbal.dat`	CVTRA01Y	50
`discgrp.dat`	CVTRA02Y	50
`custdata.dat`, `carddata.dat`, `trantype.dat`, `trancatg.dat`	CUSTREC, CVACT02Y, CVTRA03Y, CVTRA04Y	500, 150, 60, 60

Run parameters: business date `2022-07-18`, INTCALC parameter `2022071800`.

Step isolation. A candidate for INTCALC receives the oracle's POSTTRAN outputs as its account and category-balance inputs, so an INTCALC result is never penalised for a POSTTRAN error.

Outputs

One JSON file per output: a list of objects, in any order. Field names and types are fixed. Strings may omit trailing spaces. Amounts are JSON strings or numbers holding at most two decimals (`"14.56"`, `-500`).

File	Step	Fields
`posttran_transact.json`	POSTTRAN	transaction fields (below) for every posted transaction
`posttran_dalyrejs.json`	POSTTRAN	transaction fields + `reason` (int) + `reason_desc` for every rejected transaction
`posttran_acctdata.json`	POSTTRAN	account fields for every account, after posting
`posttran_tcatbal.json`	POSTTRAN	`acct_id, type_cd, cat_cd, bal` for every category balance, after posting
`intcalc_systran.json`	INTCALC	transaction fields for every interest transaction written
`intcalc_acctdata.json`	INTCALC	account fields for every account, after interest

- Transaction fields: `tran_id, type_cd, cat_cd (int), source, desc, amt, merchant_id (int), merchant_name, merchant_city, merchant_zip, card_num, orig_ts, proc_ts`

- Account fields: `acct_id (int), active_status, curr_bal, credit_limit, cash_credit_limit, open_date, expiration_date, reissue_date, curr_cyc_credit, curr_cyc_debit, addr_zip, group_id`

A run may instead leave the oracle's fixed-width dumps (`<name>.txt`). The scorer reads either form.

Optional `flags.json`: the uncertainties the pipeline declared **before** scoring, as `[{"rule": "R-INT-TRUNC", "source": "<artefact path and line>"}]` or `[{"output": "...", "key": "...", "source": "..."}]`. See PROTOCOL.md §6 for what counts as a flag.

Candidate I/O contract, layer 1 supplement (statement generation, daily transaction report)

This supplements CONTRACT.md for the two layer 1 steps. Like CONTRACT.md it fixes **formats only**: which files a run receives, which files it writes, and where each field sits on a printed line. It says nothing about how a field's value is computed or assembled (which records go into a statement or a total, how a name or an address line is built, which records are selected). That is behaviour, and it comes from the wiki.

Inputs (given per run)

Fixed-width text, one record per line, layouts as in the copybooks (CONTRACT.md).

Step	File	Copybook	Record
statement generation	`transact.dat`: the day's combined transaction file	CVTRA05Y	350
statement generation	`acctdata.dat`: accounts after the day's posting and interest	CVACT01Y	300
statement generation	`cardxref.dat`	CVACT03Y	50
statement generation	`custdata.dat`	CUSTREC	500
daily transaction report	`transact.dat`: the day's combined transaction file	CVTRA05Y	350
daily transaction report	`cardxref.dat`	CVACT03Y	50
daily transaction report	`trantype.dat`, `trancatg.dat`	CVTRA03Y, CVTRA04Y	60, 60

Run parameters: daily transaction report start date `2022-07-01` and end date `2022-07-31` (`YYYY-MM-DD`).

Step isolation. Each step receives the oracle's upstream outputs (the combined transaction file and the accounts after interest), so a result is never penalised for an upstream error.

Outputs

Each output is a text file of fixed-length lines, one line per printed line, in the order printed. Trailing spaces may be omitted. Instead of a file, a run may return the same lines as one string (lines joined by `\n`) or as a list of strings, under the file's name without extension.

File	Step	Line length
`creastmt_statements.txt`	statement generation	80
`creastmt_statements_html.txt`	statement generation	100
`tranrept_report.txt`	daily transaction report	133

Optional `assumptions.md` and `flags.json` as in CONTRACT.md.

Picture notation in the tables below, as in COBOL: `X(n)` text of n characters, left-aligned, space-padded; `9(9).99-` nine digits with leading zeros, a point, two decimals and a trailing sign (`-` or a space); `Z(9).99-` the same with leading zeros printed as spaces; `+ZZZ,ZZZ,ZZZ.ZZ` / `-ZZZ,ZZZ,ZZZ.ZZ` a sign in the first position (`+`/`-`, or space/`-`), leading zeros and their commas printed as spaces.

`creastmt_statements.txt`: one statement, lines in this order

#	Line (positions are 1-based)
0	`*` × 31, `START OF STATEMENT`, `*` × 31
1	1–75 customer name X(75); 76–80 spaces
2	1–50 address line 1 X(50); 51–80 spaces
3	1–50 address line 2 X(50); 51–80 spaces
4	1–80 address line 3 X(80)
5	`-` × 80
6	33 spaces, `Basic Details`, spaces to 80
5	`-` × 80
7	1–20 `Account ID :`; 21–40 account ID X(20); spaces to 80
8	1–20 `Current Balance :`; 21–33 current balance `9(9).99-`; spaces to 80
9	1–20 `FICO Score :`; 21–40 FICO score X(20); spaces to 80
10	`-` × 80
11	30 spaces, `TRANSACTION SUMMARY`, spaces to 80
12	`-` × 80
13	`Tran ID` padded to 16, `Tran Details` padded to 51, `Tran Amount` (80 in all)
12	`-` × 80
14	one per transaction: 1–16 transaction ID X(16); 17 space; 18–66 transaction details X(49); 67 `$`; 68–80 amount `Z(9).99-`
12	`-` × 80
14A	1–10 `Total EXP:`; 11–66 spaces; 67 `$`; 68–80 total `Z(9).99-`
15	`*` × 32, `END OF STATEMENT`, `*` × 32

`creastmt_statements_html.txt`: one statement, lines in this order

Each line is one of the fixed lines below or a fixed line with one field inserted (`{…}`), padded to 100. The transaction block repeats once per transaction; then the closing block ends the statement.

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
<meta charset="utf-8">
<title>HTML Table Layout</title>
</head>
<body style="margin:0px;">
<table  align="center" frame="box" style="width:70%; font:12px Segoe UI,sans-serif;">
<tr>
<td colspan="3" style="padding:0px 5px;background-color:#1d1d96b3;">
<h3>Statement for Account Number: {account ID X(20)}</h3>
</td>
</tr>
<tr>
<td colspan="3" style="padding:0px 5px;background-color:#FFAF33;">
<p style="font-size:16px">Bank of XYZ</p>
<p>410 Terry Ave N</p>
<p>Seattle WA 99999</p>
</td>
</tr>
<tr>
<td colspan="3" style="padding:0px 5px;background-color:#f2f2f2;">
<p style="font-size:16px">{customer name}  </p>
<p>{address line 1}  </p>
<p>{address line 2}  </p>
<p>{address line 3}  </p>
</td>
</tr>
<tr>
<td colspan="3" style="padding:0px 5px;background-color:#33FFD1; text-align:center;">
<p style="font-size:16px">Basic Details</p>
</td>
</tr>
<tr>
<td colspan="3" style="padding:0px 5px;background-color:#f2f2f2;">
<p>Account ID         : {account ID X(20)}</p>
<p>Current Balance    : {current balance 9(9).99-}</p>
<p>FICO Score         : {FICO score X(20)}</p>
</td>
</tr>
<tr>
<td colspan="3" style="padding:0px 5px;background-color:#33FFD1; text-align:center;">
<p style="font-size:16px">Transaction Summary</p>
</td>
</tr>
<tr>
<td style="width:25%; padding:0px 5px; background-color:#33FF5E; text-align:left;">
<p style="font-size:16px">Tran ID</p>
</td>
<td style="width:55%; padding:0px 5px; background-color:#33FF5E; text-align:left;">
<p style="font-size:16px">Tran Details</p>
</td>
<td style="width:20%; padding:0px 5px; background-color:#33FF5E; text-align:right;">
<p style="font-size:16px">Amount</p>
</td>
</tr>
--- transaction block, once per transaction ---
<tr>
<td style="width:25%; padding:0px 5px; background-color:#f2f2f2; text-align:left;">
<p>{transaction ID X(16)}</p>
</td>
```

```
<td style="width:55%; padding:0px 5px; background-color:#f2f2f2; text-align:left;">
<p>{transaction details X(49)}</p>
</td>
<td style="width:20%; padding:0px 5px; background-color:#f2f2f2; text-align:right;">
<p>{amount Z(9).99-}</p>
</td>
</tr>
--- closing block ---
<tr>
<td colspan="3" style="padding:0px 5px;background-color:#1d1d96b3;">
<h3>End of Statement</h3>
</td>
</tr>
</table>
</body>
</html>
```

`tranrept_report.txt`: line types

Line	Layout (positions are 1-based)
report header	1–38 `DALYREPT`; 39–79 `Daily Transaction Report`; 80–91 `Date Range: `; 92–101 start date; 102–105 `to`; 106–115 end date
blank line	spaces
column header	`Transaction ID` padded to 17, `Account ID` to 12, `Transaction Type` to 19, `Tran Category` to 35, `Tran Source` to 14, one space, `Amount` padded to 16
rule	`-` × 133
detail	1–16 transaction ID; 17 space; 18–28 account ID; 29 space; 30–31 type code; 32 `-`; 33–47 type description X(15); 48 space; 49–52 category code 9(4); 53 `-`; 54–82 category description X(29); 83 space; 84–93 source X(10); 94–97 spaces; 98–112 amount `-ZZZ,ZZZ,ZZZ.ZZ`; 113–114 spaces
page total	1–11 `Page Total`; 12–97 `.` × 86; 98–112 total `+ZZZ,ZZZ,ZZZ.ZZ`
account total	1–13 `Account Total`; 14–97 `.` × 84; 98–112 total `+ZZZ,ZZZ,ZZZ.ZZ`
grand total	1–11 `Grand Total`; 12–97 `.` × 86; 98–112 total `+ZZZ,ZZZ,ZZZ.ZZ`

A page starts with the report header, the blank line, the column header and a rule. Each page total and each account total line is followed by a rule.